Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI
Email: vtu29748@veltech.edu.in
Roll no: vtu29748
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 5_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 48

Section 1 : Coding

1. Problem Statement

Suppose you are working on a project that involves sorting a large dataset of student records based on a single criterion, such as their CGPA, age, or major. You have decided to use a modified quicksort algorithm to efficiently sort the dataset while ensuring that the sorting is stable, i.e., that records with equal values maintain their original relative order.

To achieve this, you will implement a modified version of the quicksort algorithm that maintains stability during the sorting process. Specifically, you will use a three-way partitioning scheme that sorts the records into three groups: those with values less than the pivot, those with values equal to the pivot, and those with values greater than the pivot. When the algorithm processes subarrays with equal pivot values, it uses insertion sort to maintain stability.

Write a function that implements this modified version of the quicksort algorithm and uses it to sort the student records according to one specified criterion (CGPA, age, or major). Your function should take as input a list of student records and return the sorted list in ascending order based on the chosen criterion. If two records have the same values for the sorting criterion, their relative order should be maintained in the sorted list.

Input Format

The first line contains an integer n, representing the number of student records to be sorted.

The next n groups of 4 lines each contain the details of each student:

- Line 1: Name (string)
- Line 2: CGPA (float)
- Line 3: Age (integer)
- Line 4: Major (string)

The last line contains the sorting criteria (one of three options: "CGPA", "age", or "major").

Output Format

The output displays the sorted student records in a table format with the following specifications:

Header row: "Name CGPA Age Major"

Each student record is displayed in one row with the following format:

- Name: Left-aligned, 10 characters wide
- CGPA: Left-aligned, 8 characters wide, displayed with 2 decimal places
- Age: Left-aligned, 5 characters wide
- Major: Left-aligned, 20 characters wide

The table is sorted in ascending order according to the chosen sorting criteria.

Refer to the sample outputs for the formatting specifications.

Sample Test Case

Input: 4

harish
7.8
23
ece
trello
8
21
cse
kamalesh
7
25
it
maaran
5.6
20
civil
age

Output: Name CGPA Age Major
maaran 5.60 20 civil
trello 8.00 21 cse
harish 7.80 23 ece
kamalesh 7.00 25 it

Answer

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char name[51];
    float cgpa;
    int age;
    char major[51];
    int index;
} Student;

int compare(Student a, Student b, char *criteria) {
    if (strcmp(criteria, "CGPA") == 0 || strcmp(criteria, "gpa") == 0) {
        if (a.cgpa < b.cgpa) return -1;
        if (a.cgpa > b.cgpa) return 1;
    } else if (strcmp(criteria, "age") == 0) {
        if (a.age < b.age) return -1;
```

```
    if (a.age > b.age) return 1;
  } else if (strcmp(criteria, "major") == 0) {
    int cmp = strcmp(a.major, b.major);
    if (cmp != 0) return cmp;
  }
  return a.index - b.index;
}

void insertionSort(Student arr[], int left, int right, char *criteria) {
  for (int i = left + 1; i <= right; i++) {
    Student key = arr[i];
    int j = i - 1;
    while (j >= left && compare(arr[j], key, criteria) > 0) {
      arr[j + 1] = arr[j];
      j--;
    }
    arr[j + 1] = key;
  }
}

void stableQuickSort(Student arr[], int left, int right, char *criteria) {
  if (left >= right) return;

  Student pivot = arr[right];

  Student *less = (Student *)malloc((right - left + 1) * sizeof(Student));
  Student *equal = (Student *)malloc((right - left + 1) * sizeof(Student));
  Student *greater = (Student *)malloc((right - left + 1) * sizeof(Student));

  int l = 0, e = 0, g = 0;

  for (int i = left; i <= right; i++) {
    int cmp = compare(arr[i], pivot, criteria);
    if (cmp < 0)
      less[l++] = arr[i];
    else if (cmp == 0)
      equal[e++] = arr[i];
    else
      greater[g++] = arr[i];
  }

  for (int i = 0; i < l; i++) arr[left + i] = less[i];
```

```
    for (int i = 0; i < e; i++) arr[left + l + i] = equal[i];
    for (int i = 0; i < g; i++) arr[left + l + e + i] = greater[i];

    free(less);
    free(equal);
    free(greater);

    if (l > 1) stableQuickSort(arr, left, left + l - 1, criteria);
    if (g > 1) stableQuickSort(arr, left + l + e, right, criteria);

    if (e > 1) insertionSort(arr, left + l, left + l + e - 1, criteria);
}

int main() {
    int n;
    scanf("%d", &n);
    getchar();

    Student *students = (Student *)malloc(n * sizeof(Student));

    for (int i = 0; i < n; i++) {
        fgets(students[i].name, 51, stdin);
        students[i].name[strcspn(students[i].name, "\n")] = '\0';

        scanf("%f", &students[i].cgpa);
        scanf("%d", &students[i].age);
        getchar();

        fgets(students[i].major, 51, stdin);
        students[i].major[strcspn(students[i].major, "\n")] = '\0';

        students[i].index = i;
    }

    char criteria[10];
    fgets(criteria, 10, stdin);
    criteria[strcspn(criteria, "\n")] = '\0';

    stableQuickSort(students, 0, n - 1, criteria);

    printf("Name      CGPA    Age   Major            \n");
    for (int i = 0; i < n; i++) {
```

```
        printf("%-10s %-8.2f %-5d %-20s\n",
               students[i].name,
               students[i].cgpa,
               students[i].age,
               students[i].major);
    }

    free(students);
    return 0;
}
```

Status : Partially correct **Marks : 8/10**

2. Problem Statement

Vishnu, a math enthusiast, is given a task to explore the magic of numbers. He has an array of positive integers, and his goal is to find the integer with the highest digit sum in the sorted array using the merge sort algorithm.

You have to assist Vishnu in implementing the merge sort algorithm.

Example

Input:

6

789 123 456 876 234 567

Output:

The sorted array is: 123 234 456 567 789 876

The integer with the highest digit sum is: 789

Explanation

Sorted Array Output: 123 234 456 567 789 876

The digit sum of an integer is the sum of its individual digits.

In this case, the integer 789 has the highest digit sum (7 + 8 + 9 = 24).

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line consists of N space-separated integers, representing the array elements.

Output Format

The first line prints "The sorted array is: " followed by the sorted array in ascending order.

The second line prints "The integer with the highest digit sum is: " followed by the integer with the highest-digit sum.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
123 456 789 321 654
Output: The sorted array is: 123 321 456 654 789
The integer with the highest digit sum is: 789

Answer

```
#include <stdio.h>

int digitSum(int num) {
    int sum = 0;
    while (num > 0) {
        sum += num % 10;
        num /= 10;
    }
    return sum;
}

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
```

```
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    mergeSort(arr, 0, n - 1);

    printf("The sorted array is: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    int maxSum = -1, result = -1;
    for (int i = 0; i < n; i++) {
```

```
        int sum = digitSum(arr[i]);
        if (sum > maxSum) {
            maxSum = sum;
            result = arr[i];
        }
    }

    printf("The integer with the highest digit sum is: %d\n", result);

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Pavi is working on a project to help a library manage its collection of books efficiently. As part of this project, you need to develop a program to assist the library staff in finding specific books in their extensive catalog.

The library's book collection is organized in such a way that each book is assigned a unique numerical identifier. The library staff has provided you with a list of these identifiers, and you need to write a program to help them find a book's identifier efficiently.

Therefore, you decide to implement a recursive binary search algorithm to quickly locate books based on their unique identifiers.

Input Format

The first line of input contains an integer, n, representing the number of books in the library's collection.

The second line of input consists of n space-separated integers, where each integer represents the unique identifier of a book in the library's collection.

The third line of input contains a single integer t, representing the unique identifier of the book the library staff is searching for.

Output Format

If the book with the identifier target is found in the library's collection, the output prints the index (0-based) of the book in the collection.

If the book is not found, print "t not present", where the t is the unique identifier of a book.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
12 13 14 15 16
14
Output: 2

Answer

```
#include <stdio.h>

int binarySearch(int arr[], int low, int high, int target) {
    if (low > high) return -1;

    int mid = (low + high) / 2;

    if (arr[mid] == target) return mid;
    else if (arr[mid] > target) return binarySearch(arr, low, mid - 1, target);
    else return binarySearch(arr, mid + 1, high, target);
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int t;
    scanf("%d", &t);

    int result = binarySearch(arr, 0, n - 1, t);
```

```
    if (result != -1) {
        printf("%d\n", result);
    } else {
        printf("%d not present\n", t);
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Jack is building a number analyzer tool that identifies numbers based on the sum of their digits. He is given a list of integers and wants to find the smallest number in the list whose digit sum equals a target value d.

To do this efficiently, Jack first sorts the list based on each number's digit sum and then applies binary search to find a match. Write a program to ensure that Jack can determine the correct number using this approach. If no number in the list matches the digit sum d, print -1.

Input Format

The first line of input consists of an integer n, representing the number of elements in the list.

The second line contains n space-separated integers, representing the sorted list.

The third line contains an integer d, representing the target digit sum.

Output Format

The output prints a single integer representing the smallest number from the list whose digit sum equals d, based on sorted digit sum order.

If no such number exists, print -1.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
12 23 30 48 56
3

Output: 12

Answer

```
#include <stdio.h>

int digitSum(int num) {
    int sum = 0;
    while (num > 0) {
        sum += num % 10;
        num /= 10;
    }
    return sum;
}

void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (digitSum(L[i]) < digitSum(R[j]) ||
            (digitSum(L[i]) == digitSum(R[j]) && L[i] < R[j])) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
```

```
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1, ans = -1;
    while (low <= high) {
        int mid = (low + high) / 2;
        int sum = digitSum(arr[mid]);
        if (sum == target) {
            ans = arr[mid];
            high = mid - 1; // keep searching left for smaller number
        } else if (sum < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return ans;
}

int main() {
    int n;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    int d;
    scanf("%d", &d);

    mergeSort(arr, 0, n - 1);
    int result = binarySearch(arr, n, d);

    if (result != -1) {
        printf("%d\n", result);
    } else {
        printf("-1\n");
    }
```

```
    return 0;
}
```

Status : Correct ***Marks : 10/10***

5. Problem Statement

Riya, an HR manager, needs a quick way to identify the Kth highest salary among employees for bonus distribution. To efficiently retrieve this information, she wants to use the max-heap technique.

Help Riya implement a solution that finds the Kth highest salary from the given list of salaries.

Input Format

The first line of input consists of an integer N, representing the number of employees.

The second line consists of N space-separated integers, representing the salaries of N employees.

The third line consists of an integer K, indicating the rank of the desired Kth highest salary.

Output Format

The output prints: <k>th largest Salary: <result>

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
30000 25000 40000 35000 20000 28000 29500
5
Output: 5th largest Salary: 28000

Answer

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(&arr[i], &arr[largest]);
        heapify(arr, n, largest);
    }
}

void buildMaxHeap(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
}

int kthLargest(int arr[], int n, int k) {
    buildMaxHeap(arr, n);
    for (int i = n - 1; i >= n - k + 1; i--) {
        swap(&arr[0], &arr[i]);
        heapify(arr, i, 0);
    }
    return arr[0];
}

int main() {
    int N;
    scanf("%d", &N);
```

```
    int arr[N];
    for (int i = 0; i < N; i++) {
        scanf("%d", &arr[i]);
    }
    int K;
    scanf("%d", &K);

    int result = kthLargest(arr, N, K);
    printf("%dth largest Salary: %d\n", K, result);

    return 0;
}
```

Status : Correct **Marks : 10/10**